

DSA - Week4 - Assignment

* WEEK - 4 *

4.9) @ show the result of inserting 3, 1, 4, 6, 9, 2, 5, 7 into an initially empty binary search tree.

Ans:- Here, given the values are 3, 1, 4, 6, 9, 2, 5 and 7.

also, given that tree was initially empty. So starting with an empty BST:

[]

i) inserting '3'

[3]

ii, inserting '1'

[3]
/
[1]

iii, inserting '4'

[3]
/ \
[1] [4]

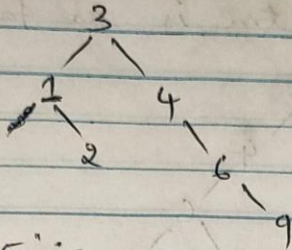
iv, inserting '6'

3
/ \
1 4
 \
 6

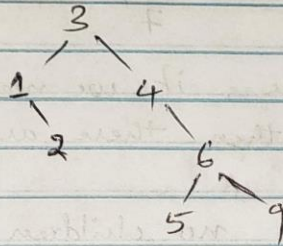
v, inserting '9'

3
/ \
1 4
 \
 6
 \
 9

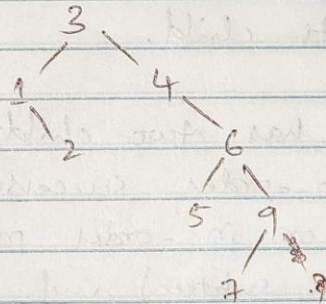
vi, Inserting '2' into the tree



vii, Inserting "5" :-



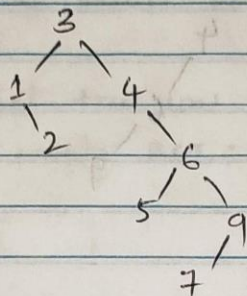
viii, Inserting "7" :-



The above tree is the resulting binary search tree. (BST) contains the values 1, 2, 3, 4, 5, 6, 7, 9. and it maintains BST property.

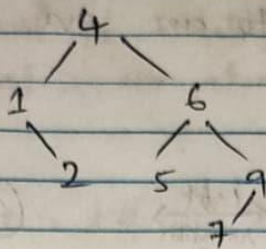
⑥ show the result of deleting the root.

Ans:- The constructed tree is as follows.



From the above tree, if we need to remove the root node (3). then there are certain rules to follow

- If the node has no children, then we can simply remove it.
- If the node has a child we can replace the node with its child.
- If the node has two children, we need to find the nodes in-order successor (smallest node in right subtree) or in-order predecessor (the largest node in the left subtree) and replace the node with that successor or predecessor.
- In this case, deleting the root (3) with two children, we need to find the in-order successor which is (4) and replace it with the node which is being deleted.
- The resulting tree is as shown below.



##

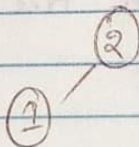
4.19) Show the result of inserting 2, 1, 4, 5, 9, 3, 6, 7 into an initially empty AVL tree.

Ans:- Given numbers : 2, 1, 4, 5, 9, 3, 6, 7

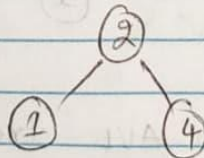
i, Inserting '2' into an ^{empty} AVL tree.



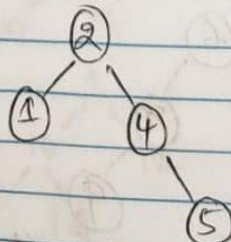
ii, Inserting '1' into an AVL tree.



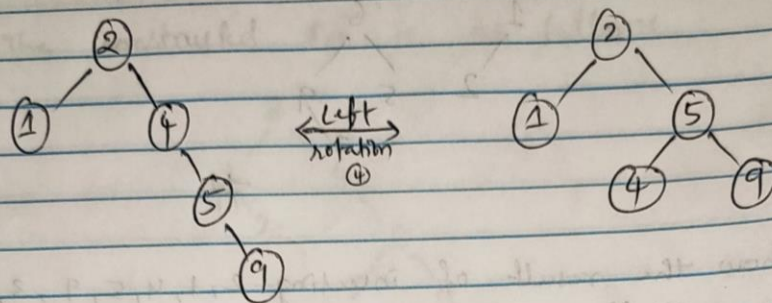
iii, Inserting '4' into an AVL tree.



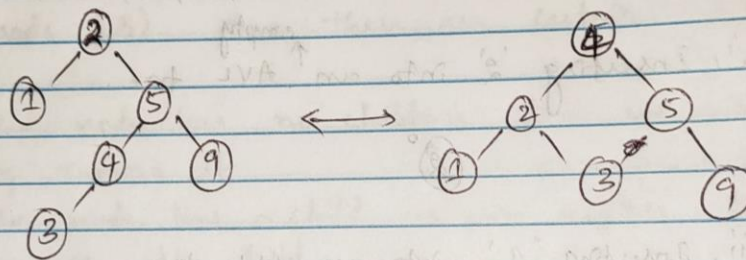
iv, Inserting '5' into an AVL tree



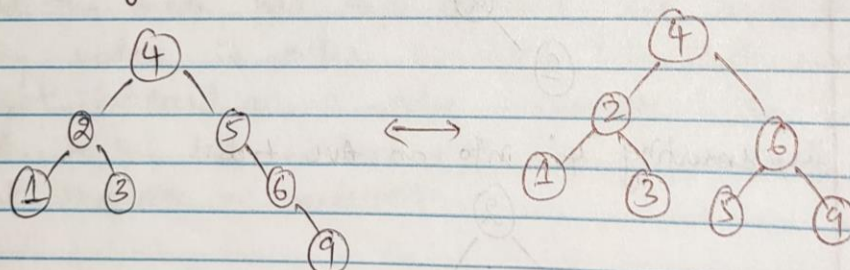
v, inserting '9' into an AVL tree



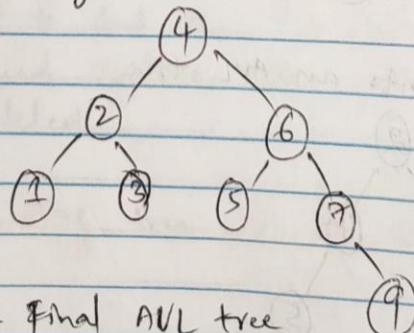
vi, Inserting '3' into an AVL tree.



vii, Inserting '6' into an AVL tree.



viii, Inserting '7' into an AVL tree.



This is the final AVL tree

#

4.22) Design a linear-time algorithm that verifies that the height information in an AVL tree is correctly maintained and that the balance property is in order. Algorithm design with AVL tree (only pseudo code is required).

Ans: -

We may create a linear-time method that traverses the tree's post-orders to ensure that the height information in an AVL tree is correctly kept and that the balance property is valid.

Here's a step-by-step description of the algorithm:

- Start by defining a recursive function, let's call it "verifyAVL, " that takes a node as input.
- Inside the "verifyAVL" function, check if the current node is null. If it is, return 0 as the height of the subtree.
- Recursively call "verifyAVL" on the left subtree of the current node and store the returned value in a variable, let's call it "leftHeight."
- Recursively call "verifyAVL" on the right subtree of the current node and store the returned value in a variable, let's call it "rightHeight."
- Calculate the height of the current node by taking the maximum of "leftHeight" and "rightHeight" and adding 1.
- Check if the absolute difference between "leftHeight" and "rightHeight" is greater than 1. If it is, return false, as the balance property is violated.
- Return the height of the current node as the result of the function.
- Finally, call the "verifyAVL" function on the root of the AVL tree to start the verification process.
- The time complexity of this algorithm is $O(n)$, where n is the number of nodes in the AVL tree, as it performs a post-order traversal of the tree.

Here's a pseudo-code representation of the algorithm:

// Took the reference from google (Brainly site) for code

```
function verifyAVL(node):  
    if node is null:  
        return 0  
    leftHeight = verifyAVL(node.left)  
    rightHeight = verifyAVL(node.right)  
    height = max (leftHeight, rightHeight) + 1
```

```
    if abs (leftHeight - rightHeight) > 1:
        return false
    return height

// Call the verifyAVL function on the root of the AVL tree
    result = verifyAVL(root)
    if result is false:
        // The height information or balance property is not correctly maintained
        // Print an error message or take appropriate action
        else:
            // The height information and balance property are correct
            // Continue with other operations or tasks
```


6.2) (a) show the result of inserting 10, 12, 1, 14, 6, 5, 8, 15, 3, 9, 7, 4, 11, 13 and 2 one at a time into an initially empty binary heap.

Ans:- To create two different heaps by inserting values 10, 12, 1, 14, 6, 5, 8, 15, 3, 9, 7, 4, 11, 13 and 2 one at a time into an initially empty binary heap.

As given in the description, we follow two different approaches.

1) Inserting Individually:- In this method, we will insert

the elements by using "insert()" function, to ensure that each element is inserted into its proper location & to maintain heap order property.

→ Insert 10 :-

10

→ Insert 12 :-

10

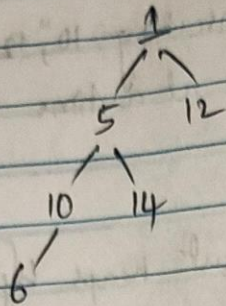
12

→ Insert 1 :-

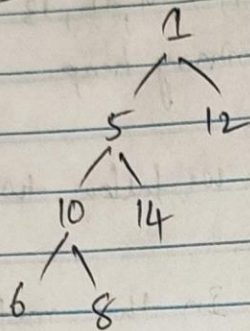
1
10 12

→ Insert 14 :-

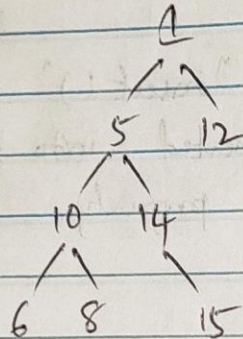
→ Insert 5 :-



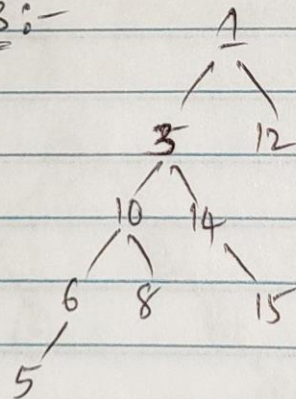
→ Insert 8 :-



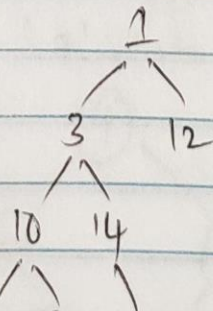
→ Insert 15 :-



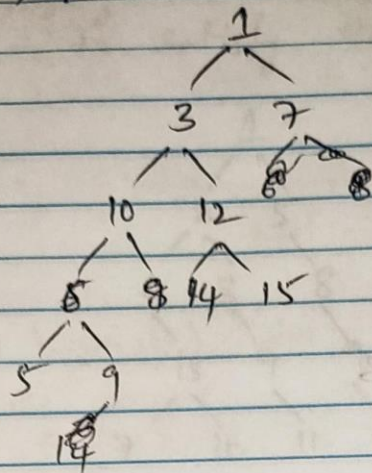
→ Insert 3 :-



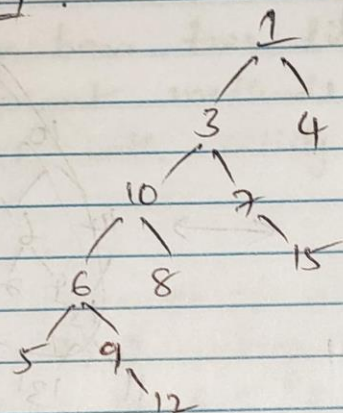
→ Insert 9 :-



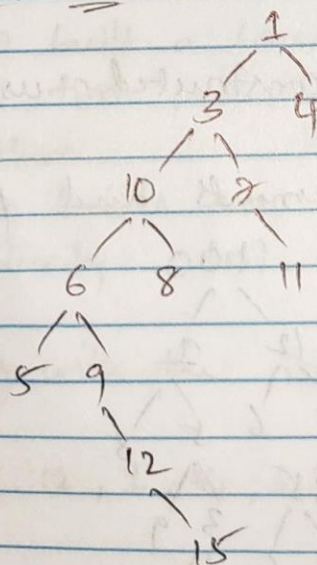
→ Insert 7 :-



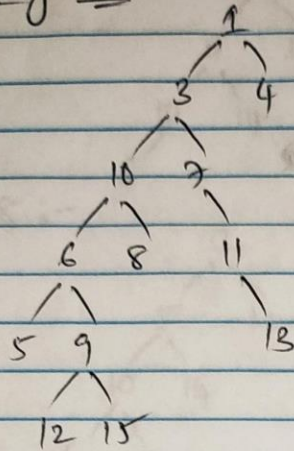
→ Insert 4 :-



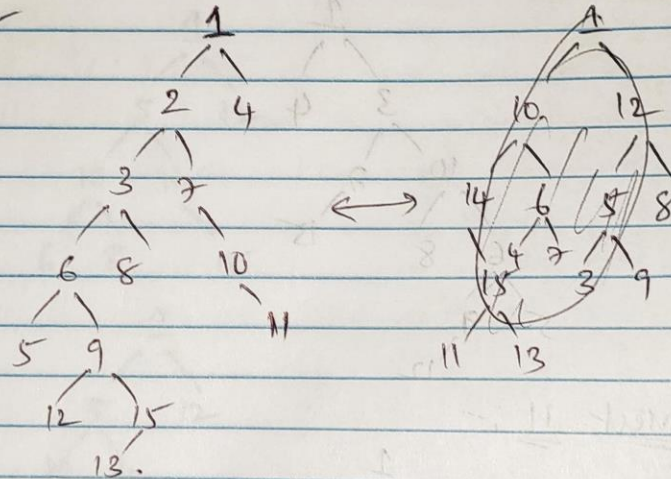
→ Insert 11 :-



→ Inserting 13 :-



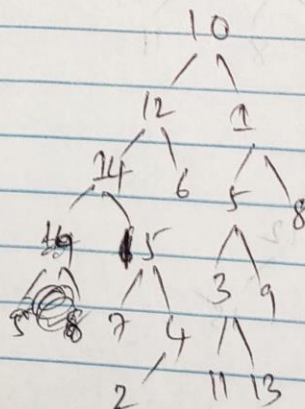
→ Insert 21 :-



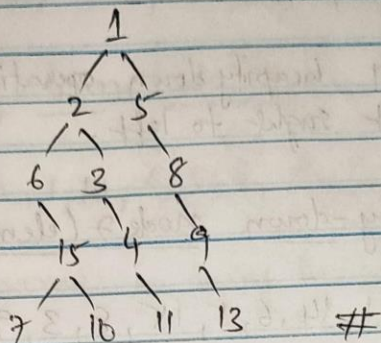
Here, the above heap is constructed using Insert (1)

Approach: 2 :- Inserting all elements and performing Build heap.

→ Inserting all at a time



→ After applying the build heap() function, the heap will be reordered to satisfy the heap order property.



→ Now we have two different heaps created by using two different methods, The heap structure is different but they both satisfy the heap order property. #

6.2)

(b) show the result of using a linear time algorithm to build a binary heap using the same input which is -
⇒ 10, 12, 1, 14, 6, 5, 8, 15, 3, 9, 7, 4, 11, 13, and 2.

Ans:- To build a binary heap using the given input in linear time, we can use linear time heap construction algorithm.

→ This builds an heap from an array using linear time complexity $O(n)$. where, n - no. of elements.

starting with the input array :-

10, 12, 1, 14, 6, 5, 8, 15, 3, 9, 7, 4, 11, 13, 2

→ Here, we need to start with the last non-leaf node and move upto the root of the array, by performing the 'heapify-down' operation at each step.

→ Need to apply heapify down operation at each non-leaf node from ~~left~~ right to left.

Step 1:- Heapify down node 7 (element 8):-

10, 12, 1, 14, 6, 5, 15, 8, 3, 9, 7, 4, 11, 13, 2

Step 2:- Heapify down node 6:-

10, 12, 1, 14, 6, 15, 5, 8, 3, 9, 7, 4, 11, 13, 2

Step 3:- Heapify down node 5:-

10, 12, 1, 14, 15, 6, 5, 8, 3, 9, 7, 4, 11, 13, 2

Step 4:- Heapify down node 4:-

10, 12, 1, 15, 14, 6, 5, 8, 3, 9, 7, 4, 11, 13, 2

Step 5:- Heapify down node 3:-

10, 12, 15, 1, 14, 6, 5, 8, 3, 9, 7, 4, 11, 13, 2

Step 6:- Heapify down node 2:-

10, 15, 12, 1, 14, 6, 5, 8, 3, 9, 7, 4, 11, 13, 2

Step 7:- Heapify-down node 2:-

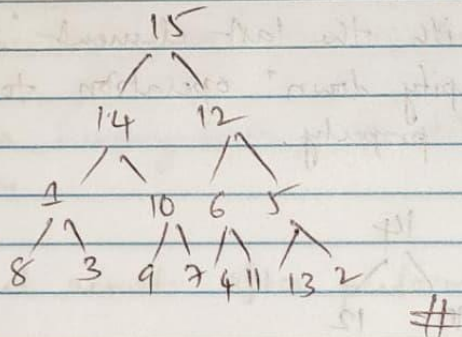
15, 14, 12, 1, 10, 6, 5, 8, 3, 9, 7, 4, 11, 13, 2

Step 8:- Heapify-down node 0:-

15, 14, 12, 1, 10, 6, 5, 8, 3, 9, 7, 4, 11, 13, 2

→ The entire array is being heapified into binary heap.

The result is:-

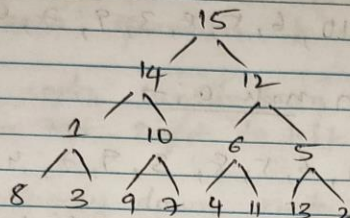


6.3) Show the result of performing three deleteMin operations in the heap of the previous exercise.

Ans:- To perform the three "deleteMin" operations on the binary heap from the previous exercise, we need to remove the root element from the heap repeatedly for three times and need to adjust it with the another element.

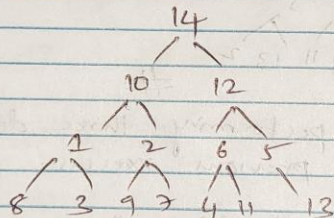
→ The below are the steps of performing deleteMin operation

* Initial binary heap:-



→ DeleteMin operation 1:-

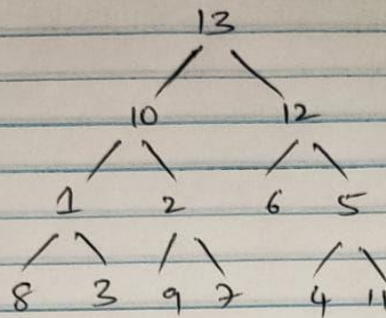
- Removes the root node (15) from the heap
- Replaces it with the last element "2" in the heap
- performs "heapify-down" operation to restore the max heap property.



* DeleteMin step operation 2:-

- Removes the root node (14)
- Replaces it with the last element 13
- then performs heapify down operation to restore the max heap property.

0. #



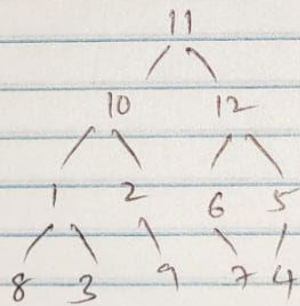
→ Delete Min operation :-

→ Removes the minimum element which is root node (13).

→ Replaces it with the last element "11".

→ performs heapify-down operation to restore max heap property

The final result after performing delete min operation.



→ After three "delete min" operations, the binary heap is modified accordingly and the resulting heap satisfies the max heap property

#/

